

P3785R0

Library Wording Changes for Defaulted Postfix Increment and Decrement Operations

Published Proposal, 2025-07-12

Authors:

[Matthew Taylor](#)

[Alex \(Waffl3x\)](#)

Audience:

LWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

Following design-approval of P3668 Defaulting Postfix Increment and Decrement Operations, it would be possible to significantly shrink the standard library wording by defaulting those operations where appropriate. This paper proposes this as an editorial change for LWG review.

Table of Contents

1 Motivation and Scope

2 Proposal

2.1 Simple Defaultable Operations

2.2 Postfix Operations Requiring LWG Input

2.3 Container Iterators and Named Requirements

3 Proposed Wording

3.1 Simple Defaultable Operations

3.2 Wording for Operators with Preconditions or Constraints

3.3 Wording for Operators with Compile-Time Branching

3.4 Wording for Operators with both Preconditions and Compile-Time Branching:

3.5 Wording for Operators with Transitive Preconditions

References

Informative References

§ 1. Motivation and Scope

[P3668] proposes explicitly defaulting postfix increment and decrement operations, giving them a definition equivalent to:

```
C operator++(C& c, int){
    C copy = C(c);
    ++c;
    return copy;
}
```

This allows users to write shorter code using this sensible default with a universally-accepted meaning. The standard library also defines many objects with postfix increment and decrement operations, and each one is defined in full with code which is equivalent to the above definition. For example, `std::regex_iterator` defines its postfix increment operation as follows:

```
regex_iterator operator++(int);  
9 Effects: As if by:  
  
    regex_iterator tmp = *this;  
    ++(*this);  
    return tmp;
```

This pattern of explicitly defining the default meaning of the postfix operation is repeated on every iterator which exhibits the default behaviour in the standard. While this has been necessary up to this point, with P3668 design-approved for C++29, we can change this. We are able to mark these operations as defaulted, which allows us to keep their meanings clear while eliminating many unnecessary boilerplate definitions from the standard document.

We consider this a strictly editorial change to the specification; and do not anticipate that implementations will have to update around this paper. The semantics of every affected overload after this proposal are identical to before it; however the specification for them is permitted to be shorter and simpler.

2. Proposal

The authors have identified 53 candidate operations in the standard which may be defaulted. Most of these are simple: they are a single postfix increment or decrement operator which has behaviour equivalent to the defaulted definition which can be immediately replaced. There are also some which we highlight for specific consideration as they have additional properties which LWG may prefer not to word as defaulted. The authors are not library wording experts, so defer such decisions to LWG.

2.1. Simple Defaultable Operations

The following table lists all standard library postfix operations which are specified as exactly equivalent to the canonical definition. This expressly does not include any iterators which have any additional complexities which might raise wording questions if they were instead specified as = default. Those are covered in the next section.

NOTE: We include the C++26 simd operators, integrating changes from [P3480] and [P3691] as voted on at the Sofia meeting. Editorial changes to the wording as quoted in those papers may be made as they are merged into the standard, however the underlying design shall remain consistent.

Standard Library Operation	Specified in	Specified
<code>move_iterator::operator--(int)</code>	[move.iter.nav]	As if by: <code>move_iterator tmp --current; return tmp;</code>
<code>std::basic_const_iterator::operator++(int)</code>	[const.iterators.ops]	Equivalent to: <code>auto tmp = *this; ++*this; return tmp;</code>
<code>std::basic_const_iterator::operator--(int)</code>	[const.iterators.ops]	Equivalent to: <code>auto tmp = *this; --*this; return tmp;</code>

<code>std::counted_iterator::operator--(int)</code>	[counted_iterator.nav]	Equivalent to: <pre>counted_iterator --*this; return tmp;</pre>
<code>std::regex_iterator::operator++(int)</code>	[re.regiter.iter]	As if by: <pre>regex_iterator tm ++(*this); return tmp;</pre>
<code>std::regex_token_iterator::operator++(int)</code>	[re.tokiter.incr]	<i>Effects:</i> Constructs a copy, then calls <code>++(*this)</code> . <i>Returns:</i> <code>tmp</code> .
<code>std::ranges::iota_view::operator++(int)</code>	[range.iota.iterator]	Equivalent to: <pre>auto tmp = *this; ++*this; return tmp;</pre>
<code>std::ranges::iota_view::operator--(int)</code>	[range.iota.iterator]	Equivalent to: <pre>auto tmp = *this; --*this; return tmp;</pre>
<code>std::ranges::repeat_view::iterator::operator++(int)</code>	[range.repeat.iterator]	Equivalent to: <pre>auto tmp = *this; ++*this; return tmp;</pre>
<code>std::ranges::filter_view::iterator::operator++(int)</code>	[range.filter.iterator]	Equivalent to: <pre>auto tmp = *this; ++*this; return tmp;</pre>
<code>std::ranges::filter_view::iterator::operator--(int)</code>	[range.filter.iterator]	Equivalent to: <pre>auto tmp = *this; --*this; return tmp;</pre>
<code>std::ranges::transform_view::iterator::operator++(int)</code>	[range.transform.iterator]	Equivalent to: <pre>auto tmp = *this; ++*this; return tmp;</pre>
<code>std::ranges::transform_view::iterator::operator--(int)</code>	[range.transform.iterator]	Equivalent to: <pre>auto tmp = *this; --*this; return tmp;</pre>
<code>std::ranges::chunk_view::iterator::operator--(int)</code> for forward ranges	[range.chunk.fwd.iter]	Equivalent to: <pre>auto tmp = *this; --*this; return tmp;</pre>

<code>std::ranges::join_view::iterator::operator++(int)</code>	[range.join.iterator]	Equivalent to: <pre>auto tmp = *this; ++*this; return tmp;</pre>
<code>std::ranges::join_view::iterator::operator--(int)</code>	[range.join.iterator]	Equivalent to: <pre>auto tmp = *this; --*this; return tmp;</pre>
<code>std::ranges::join_with_view::iterator::operator++(int)</code>	[range.join.with.iterator]	Equivalent to: <pre>iterator tmp = *t ++*this; return tmp;</pre>
<code>std::ranges::join_with_view::iterator::operator--(int)</code>	[range.join.with.iterator]	Equivalent to: <pre>iterator tmp = *t --*this; return tmp;</pre>
<code>std::ranges::split_view::iterator::operator++(int)</code>	[range.split.iterator]	Equivalent to: <pre>auto tmp = *this; ++*this; return tmp;</pre>
<code>std::ranges::elements_view::iterator::operator++(int)</code>	[range.elements.iterator]	Equivalent to: <pre>auto tmp = *this ++current_; return tmp;</pre> NB: <code>operator++()</code> is s <pre>++current_; return *this;</pre>
<code>std::ranges::elements_view::iterator::operator--(int)</code>	[range.elements.iterator]	Equivalent to: <pre>auto tmp = *this --current_; return tmp;</pre> NB: <code>operator--()</code> is s <pre>--current_; return *this;</pre>
<code>std::ranges::enumerate_view::iterator::operator++(int)</code>	[range.enumerate.iterator]	Equivalent to: <pre>auto tmp = *this ++*this; return tmp;</pre>

<code>std::ranges::enumerate_view::iterator::operator--(int)</code>	[range.enumerate.iterator]	Equivalent to: <pre>auto tmp = *this; --*this; return tmp;</pre>
<code>std::ranges::zip_view::iterator::operator++(int)</code>	[range.zip.iterator]	Equivalent to: <pre>auto tmp = *this; ++*this; return tmp;</pre>
<code>std::ranges::zip_view::iterator::operator--(int)</code>	[range.zip.iterator]	Equivalent to: <pre>auto tmp = *this; --*this; return tmp;</pre>
<code>std::ranges::zip_transform_view::iterator::operator++(int)</code>	[range.zip.transform.iterator]	Equivalent to: <pre>auto tmp = *this; ++*this; return tmp;</pre>
<code>std::ranges::zip_transform_view::iterator::operator--(int)</code>	[range.zip.transform.iterator]	Equivalent to: <pre>auto tmp = *this; --*this; return tmp;</pre>
<code>std::ranges::adjacent_transform_view::iterator::operator++(int)</code>	[range.adjacent.transform.iterator]	Equivalent to: <pre>auto tmp = *this; ++*this; return tmp;</pre>
<code>std::ranges::adjacent_transform_view::iterator::operator--(int)</code>	[range.adjacent.transform.iterator]	Equivalent to: <pre>auto tmp = *this; --*this; return tmp;</pre>
<code>std::ranges::chunk_by_view::iterator::operator--(int)</code>	[range.chunk.by.iter]	Equivalent to: <pre>auto tmp = *this; --*this; return tmp;</pre>
<code>std::ranges::stride_view::iterator::operator++(int)</code>	[range.stride.iterator]	Equivalent to: <pre>auto tmp = *this; ++*this; return tmp;</pre>
<code>std::ranges::stride_view::iterator::operator--(int)</code>	[range.stride.iterator]	Equivalent to: <pre>auto tmp = *this; --*this; return tmp;</pre>

<code>std::ranges::cartesian_product_view::iterator::operator++(int)</code>	[range.cartesian.iterator]	Equivalent to: <pre>auto tmp = *this; ++*this; return tmp;</pre>
<code>std::ranges::cartesian_product_view::iterator::operator--(int)</code>	[range.cartesian.iterator]	Equivalent to: <pre>auto tmp = *this; --*this; return tmp;</pre>
<code>std::simd::simd_iterator::operator++(int)</code>	[simd.iterator]	Equivalent to: <pre>simd_iterator tmp *this += 1; return tmp;</pre> NB: <code>operator++()</code> is s Equivalent to: <code>return</code>
<code>std::simd::simd_iterator::operator--(int)</code>	[simd.iterator]	Equivalent to: <pre>simd_iterator tmp *this -= 1; return tmp;</pre> NB: <code>operator--()</code> is s Equivalent to: <code>return</code>

2.2. Postfix Operations Requiring LWG Input

This table lists postfix operations which have equivalent semantics to the default behaviour, but which have complexities in the specification, and so may require some input from library wording experts. We have intentionally made this table as conservative as possible and intentionally list things which we believe will be uncontentious to change; but explicitly want to ensure that any wording questions, no matter how trivial, are noted.

To define the terms we use to describe why we believe an operator belongs in this table:

- Compile-time branching - the specification either uses `if constexpr` or wording to state that the behaviour of a function is equivalent to the default in some cases (typically when operating on a forward range), but not others. This could be remedied by introducing a constrained defaulted overload.
- Precondition/Constraint - the postfix operator overload lists a formal precondition or constraint. It is not clear whether LWG would prefer to avoid defaulting functions with listed preconditions or constraints.
- Transitive precondition - the postfix operator overload itself has no preconditions or postconditions; however one of the dependent operations (typically the prefix) does list preconditions or postconditions.

Standard Library Operation	Specified in	Specified as
<code>std::istream_iterator::operator++(int)</code>	[istream.iterator.ops]	Preconditions: <code>in_stream != null</code> Effects: Equivalent to: <pre>istream_iterator tmp = *this; ++*this; return tmp;</pre>
<code>std::simd::basic_mask::operator++(int)</code>	[simd.class]	Constraints: <code>requires (value_ty</code> <code>) is true.</code> Effects: Increments every element by 1 Returns: A copy of <code>*this</code> before incr

		NB: <code>operator++()</code> is specified as: <i>Effects</i>: Increments every element by 1. <i>Returns</i>: <code>*this</code>.
<code>std::simd::basic_mask::operator--(int)</code>	[simd.class]	<i>Constraints</i>: <code>requires (value_type::is_integral && !is_pointer<value_type>())</code> is true. <i>Effects</i>: Decrements every element by 1. <i>Returns</i>: A copy of <code>*this</code> before decrement. NB: <code>operator--()</code> is specified as: <i>Effects</i>: Decrements every element by 1. <i>Returns</i>: <code>*this</code>.
<code>move_iterator::operator++(int)</code>	[move.iter.nav]	<i>Effects</i>: If Iterator models forward_iterator, equivalent to: <pre>move_iterator tmp = *this; ++current; return tmp;</pre> Otherwise, equivalent to <code>++current</code>.
<code>std::ranges::lazy_split_view::outer_iterator::operator++(int)</code>	[range.lazy.split.outer]	<pre>if constexpr (forward_iterator) auto tmp = *this; ++*this; return tmp; else ++*this; }</pre>
<code>std::ranges::lazy_split_view::inner_iterator::operator++(int)</code>	[range.lazy.split.inner]	<pre>if constexpr (forward_iterator) auto tmp = *this; ++*this; return tmp; else ++*this; }</pre>
<code>std::common_iterator::operator++(int)</code>	[common.iter.nav]	<i>Preconditions</i>: <code>holds_alternative<T>(*this)</code> true. <i>Effects</i>: If I models forward_iterator, equivalent to: <pre>common_iterator tmp = *this; ++*this; return tmp;</pre> Otherwise: ...
<code>std::ranges::repeat_view::iterator::operator--(int)</code>	[range.repeat.iterator]	Equivalent to: <pre>auto tmp = *this; --*this; return tmp;</pre>
<code>std::counted_iterator::operator++(int)</code>	[counted.iterator.nav]	Equivalent to: <pre>counted_iterator tmp = *this; ++*this; return tmp;</pre>

<code>std::ranges::concat_view::iterator::operator++(int)</code>	[range.concat.iterator]	Equivalent to: <pre>auto tmp = *this; ++*this; return tmp;</pre>
<code>std::ranges::concat_view::iterator::operator--(int)</code>	[range.concat.iterator]	Equivalent to: <pre>auto tmp = *this; --*this; return tmp;</pre>
<code>std::ranges::adjacent_view::iterator::operator++(int)</code>	[range.adjacent.iterator]	Equivalent to: <pre>auto tmp = *this; ++*this; return tmp;</pre>
<code>std::ranges::adjacent_view::iterator::operator--(int)</code>	[range.adjacent.iterator]	Equivalent to: <pre>auto tmp = *this; --*this; return tmp;</pre>
<code>std::ranges::chunk_view::iterator::operator++(int)</code> for forward ranges	[range.chunk.fwd.iter]	Equivalent to: <pre>auto tmp = *this; ++*this; return tmp;</pre>
<code>std::ranges::slide_view::iterator::operator++(int)</code>	[range.slide.iterator]	Equivalent to: <pre>auto tmp = *this; ++*this; return tmp;</pre>
<code>std::ranges::slide_view::iterator::operator--(int)</code>	[range.slide.iterator]	Equivalent to: <pre>auto tmp = *this; --*this; return tmp;</pre>
<code>std::ranges::chunk_by_view::iterator::operator++(int)</code>	[range.chunk.by.iter]	Equivalent to: <pre>auto tmp = *this; ++*this; return tmp;</pre>

2.3. Container Iterators and Named Requirements

The iterators for standard library containers are not specified in terms of specific functions in the same way the above operations are, but instead as being some type which meets one or more named requirements or iterator concepts. For example, `std::vector::iterator` must meet *Cpp17RandomAccessIterator* and model `std::contiguous_iterator`; and the standard defers to these rather than providing a dedicated section for `std::vector::iterator::operator++(int)`.

We do not propose changing the contents of the named requirements tables or the wording for the standard iterator concepts. The purpose of this proposal is a net simplification of standard library wording and we feel that this would work against that goal. The most we would suggest is adding a note to the table to state that a defaulted operation is valid for that case, but we defer to LWG's judgement on whether it is necessary.

3. Proposed Wording

3.1. Simple Defaultable Operations

Modify §24.5.3 [const.iterators] as follows:

```
constexpr basic_const_iterator& operator++();  
constexpr void operator++(int);  
constexpr basic_const_iterator operator++(int) requires forward_iterator<Iterator> = default;
```

```
constexpr basic_const_iterator& operator--() requires bidirectional_iterator<Iterator>;  
constexpr basic_const_iterator operator--(int) requires bidirectional_iterator<Iterator>;
```

[...]

```
constexpr basic_const_iterator operator++(int) requires forward_iterator<Iterator>;
```

¹⁰~~Effects: Equivalent to:~~

```
auto tmp = *this;  
++*this;  
return tmp;
```

[...]

```
constexpr basic_const_iterator operator--(int) requires bidirectional_iterator<Iterator>;
```

¹²~~Effects: Equivalent to:~~

```
auto tmp = *this;  
--*this;  
return tmp;
```

Modify §24.5.4 [move.iterators] as follows:

```
constexpr move_iterator& operator++();  
constexpr auto operator++(int);  
constexpr move_iterator& operator--();  
constexpr move_iterator operator--(int) = default;
```

[...]

```
constexpr move_iterator operator--(int);
```

⁶~~Effects: As if by:~~

```
move_iterator tmp = *this;  
--current;  
return tmp;
```

Modify §24.5.7 [iterators.counted] as follows:

```
constexpr counted_iterator& operator--()
    requires bidirectional_iterator<I>;
constexpr counted_iterator operator--(int)
    requires bidirectional_iterator<I> = default;

[...]
```

~~constexpr counted_iterator operator--(int) requires bidirectional_iterator<I>~~

⁷ ~~Effects:~~ Equivalent to:

```
counted_iterator tmp = *this;
--*this;
return tmp;
```

Modify §25.6.4.3 [range.iota.iterator] as follows:

```
constexpr void operator++(int);
constexpr iterator operator++(int) requires incrementable<W> = default;

constexpr iterator& operator--() requires decrementable<W>;
constexpr iterator operator--(int) requires decrementable<W> = default;

[...]
```

~~constexpr iterator operator++(int) requires incrementable<W>;~~

⁸ ~~Effects:~~ Equivalent to:

```
auto tmp = *this;
++*this;
return tmp;
```

[...]

~~constexpr iterator operator--(int) requires decrementable<W>;~~

⁹ ~~Effects:~~ Equivalent to:

```
auto tmp = *this;
--*this;
return tmp;
```

Modify §25.6.5.3 [range.repeat.iterator] as follows:

```
constexpr iterator& operator++();
constexpr iterator operator++(int) = default;

[...]
```

~~constexpr iterator operator++(int);~~

⁶ ~~Effects:~~ Equivalent to:

```
auto tmp = *this;
++*this;
return tmp;
```

Modify §25.7.8.3 [range.filter.iterator] as follows:

```
constexpr iterator& operator++();
constexpr void operator++(int);
constexpr iterator operator++(int) requires forward_range<V> = default;

constexpr iterator& operator--() requires bidirectional_range<V>;
constexpr iterator operator--(int) requires bidirectional_range<V> = default;

[...]
```

```
constexpr iterator operator++(int) requires forward_range<V>;
# Effects: Equivalent to:

auto tmp = *this;
++*this;
return tmp;

[...]
```

```
constexpr iterator operator--(int) requires bidirectional_range<V>;
# Effects: Equivalent to:

auto tmp = *this;
--*this;
return tmp;
```

Modify §25.7.9.3 [range.transform.iterator] as follows:

```
constexpr iterator& operator++();
constexpr void operator++(int);
constexpr iterator operator++(int) requires forward_range<Base> = default;

constexpr iterator& operator--() requires bidirectional_range<Base>;
constexpr iterator operator--(int) requires bidirectional_range<Base> = default;

[...]
```

```
constexpr iterator operator++(int) requires forward_range<Base>;
# Effects: Equivalent to:

auto tmp = *this;
++*this;
return tmp;

[...]
```

```
constexpr iterator operator--(int) requires bidirectional_range<Base>;
# Effects: Equivalent to:

auto tmp = *this;
--*this;
return tmp;
```

Modify §25.7.9.7 [range.chunk.fwd.iter] as follows:

```
constexpr iterator& operator--() requires bidirectional_range<Base>;  
constexpr iterator operator--(int) requires bidirectional_range<Base> = default;
```

[...]

```
constexpr iterator operator--(int) requires bidirectional_range<Base>;
```

~~¹⁵ Effects: Equivalent to:~~

```
auto tmp = *this;  
--*this;  
return tmp;
```

Modify §25.7.14.3 [range.join.iterator] as follows:

```
constexpr iterator& operator++();  
constexpr void operator++(int);  
constexpr iterator operator++(int)  
    requires ref-is-glvalue && forward_range<Base> &&  
        forward_range<range_reference_t<Base>> = default;
```

```
constexpr iterator& operator--()  
    requires ref-is-glvalue && bidirectional_range<Base> &&  
        bidirectional_range<range_reference_t<Base>> &&  
        common_range<range_reference_t<Base>>;
```

```
constexpr iterator operator--(int)  
    requires ref-is-glvalue && bidirectional_range<Base> &&  
        bidirectional_range<range_reference_t<Base>> &&  
        common_range<range_reference_t<Base>> = default
```

[...]

```
constexpr iterator operator++(int) requires ref is glvalue && forward_range<Base> &&  
    forward_range<range_reference_t<Base>>;
```

~~¹⁵ Effects: Equivalent to:~~

```
auto tmp = *this;  
++*this;  
return tmp;
```

[...]

```
constexpr iterator operator--(int) requires ref is glvalue && bidirectional_range<Base>  
&& bidirectional_range<range_reference_t<Base>> && common_range<range_reference_t<Base>>;
```

~~¹⁵ Effects: Equivalent to:~~

```
auto tmp = *this;  
--*this;  
return tmp;
```

Modify §25.7.15.3 [range.join.with.iterator] as follows:

```
constexpr iterator& operator++();
constexpr void operator++(int);
constexpr iterator operator++(int)
    requires ref-is-glvalue && forward_iterator<OuterIter> &&
        forward_iterator<InnerIter> = default;

constexpr iterator& operator--()
    requires ref-is-glvalue && bidirectional_range<Base> &&
        bidirectional-common<InnerBase> && bidirectional-common<PatternBase>;
constexpr iterator operator--(int)
    requires ref-is-glvalue && bidirectional_range<Base> &&
        bidirectional-common<InnerBase> && bidirectional-common<PatternBase> = default;

[...]
```

~~constexpr iterator operator++(int) requires ref is glvalue && forward_iterator<OuterIter>~~
~~&& forward_iterator<InnerIter>;~~

~~¹⁵ Effects: Equivalent to:~~

```
iterator tmp = *this;  
++*this;  
return tmp;
```

[...]

~~constexpr iterator operator--(int) requires ref is glvalue && bidirectional_range<Base>~~
~~&& bidirectional-common<InnerBase> && bidirectional-common<PatternBase>;~~

~~¹⁷ Effects: Equivalent to:~~

```
iterator tmp = *this;  
--*this;  
return tmp;
```

Modify §25.7.17.3 [range.split.iterator] as follows:

```
constexpr iterator& operator++();
constexpr iterator operator++(int) = default;

[...]
```

~~constexpr iterator operator++(int);~~

~~⁵ Effects: Equivalent to:~~

```
iterator tmp = *this;  
++*this;  
return tmp;
```

Modify §25.7.23.3 [range.elements.iterator] as follows:

```
constexpr iterator& operator++();
constexpr void operator++(int);
constexpr iterator operator++(int) requires forward_range<Base> = default;

constexpr iterator& operator--() requires bidirectional_range<Base>;
constexpr iterator operator--(int) requires bidirectional_range<Base> = default;

[...]
```

~~constexpr iterator operator++(int) requires forward_range<Base>;~~
~~*#0 Effects:* Equivalent to:~~

```
auto tmp = *this;  
++current_;  
return tmp;
```

[...]

~~constexpr iterator operator--(int) requires bidirectional_range<Base>;~~
~~*#2 Effects:* Equivalent to:~~

```
auto tmp = *this;  
--current_;  
return tmp;
```

Modify §25.7.24.3 [range.enumerate.iterator] as follows:

```
constexpr iterator& operator++();
constexpr void operator++(int);
constexpr iterator operator++(int) requires forward_range<Base> = default;

constexpr iterator& operator--() requires bidirectional_range<Base>;
constexpr iterator operator--(int) requires bidirectional_range<Base> = default;

[...]
```

~~constexpr iterator operator++(int) requires forward_range<Base>;~~
~~*# Effects:* Equivalent to:~~

```
auto temp = *this;  
++*this;  
return temp;
```

[...]

~~constexpr iterator operator--(int) requires bidirectional_range<Base>;~~
~~*# Effects:* Equivalent to:~~

```
auto temp = *this;  
--*this;  
return temp;
```

Modify §25.7.25.3 [range.zip.iterator] as follows:

```
constexpr void operator++(int);
constexpr iterator operator++(int) requires all-forward<Const, Views...> = default;

constexpr iterator& operator--() requires all-bidirectional<Const, Views...>;
constexpr iterator operator--(int) requires all-bidirectional<Const, Views...> = default;

[...]
```

~~constexpr iterator operator++(int) requires all-forward<Const, Views...>;~~
~~# Effects: Equivalent to:~~

```
auto tmp = *this;
++*this;
return tmp;
```

[...]

~~constexpr iterator operator--(int) requires all-bidirectional<Const, Views...>;~~
~~# Effects: Equivalent to:~~

```
auto tmp = *this;
--*this;
return tmp;
```

Modify §25.7.26.3 [range.zip.transform.iterator] as follows:

```
constexpr iterator& operator++();
constexpr void operator++(int);
constexpr iterator operator++(int) requires forward_range<Base> = default;

constexpr iterator& operator--() requires bidirectional_range<Base>;
constexpr iterator operator--(int) requires bidirectional_range<Base> = default;

[...]
```

~~constexpr iterator operator++(int) requires forward_range<Base>;~~
~~# Effects: Equivalent to:~~

```
auto tmp = *this;
++*this;
return tmp;
```

[...]

~~constexpr iterator operator--(int) requires bidirectional_range<Base>;~~
~~# Effects: Equivalent to:~~

```
auto tmp = *this;
--*this;
return tmp;
```

Modify §25.7.28.3 [range.adjacent.transform.iterator] as follows:

```
constexpr iterator& operator++();  
constexpr iterator operator++(int) = default;  
constexpr iterator& operator--() requires bidirectional_range<Base>;  
constexpr iterator operator--(int) requires bidirectional_range<Base> = default;
```

[...]

~~constexpr iterator operator++(int);~~

⁷ ~~Effects:~~ Equivalent to:

```
auto tmp = *this;  
++*this;  
return tmp;
```

[...]

~~constexpr iterator operator--(int) requires bidirectional_range<Base>;~~

⁹ ~~Effects:~~ Equivalent to:

```
auto tmp = *this;  
--*this;  
return tmp;
```

Modify §25.7.31.3 [range.chunk.by.iter] as follows:

```
constexpr iterator& operator--() requires bidirectional_range<V>;  
constexpr iterator operator--(int) requires bidirectional_range<V> = default;
```

[...]

~~constexpr iterator& operator--() requires bidirectional_range<V>;~~

⁹ ~~Effects:~~ Equivalent to:

```
auto tmp = *this;  
--*this;  
return tmp;
```


Modify §25.7.32.3 [range.stride.iterator] as follows:

```
constexpr iterator& operator++();
constexpr iterator operator++(int) = default;
constexpr iterator& operator--() requires bidirectional_range<Base>;
constexpr iterator operator--(int) requires bidirectional_range<Base> = default;
```

[...]

~~constexpr iterator operator++(int);~~

~~¹⁰ Effects: Equivalent to:~~

```
auto tmp = *this;
++*this;
return tmp;
```

[...]

~~constexpr iterator operator--(int) requires bidirectional_range<Base>;~~

~~¹² Effects: Equivalent to:~~

```
auto tmp = *this;
--*this;
return tmp;
```

Modify §25.7.33.3 [range.cartesian.iterator] as follows:

```
constexpr iterator& operator++();
constexpr void operator++(int);
constexpr iterator operator++(int) requires forward_range<maybe-const<Const, First>> = default;
```

```
constexpr iterator& operator--()
    requires cartesian-product-is-bidirectional<Const, First, Vs...>;
constexpr iterator operator--(int)
    requires cartesian-product-is-bidirectional<Const, First, Vs...> = default;
```

[...]

~~constexpr iterator operator++(int) requires forward_range<maybe-const<Const, First>>;~~

~~¹⁴ Effects: Equivalent to:~~

```
auto tmp = *this;
++*this;
return tmp;
```

[...]

~~constexpr iterator operator--(int) requires cartesian-product-is-bidirectional<Const, First, Vs...>;~~

~~¹⁶ Effects: Equivalent to:~~

```
auto tmp = *this;
--*this;
return tmp;
```

Modify §28.6.11.1 [re.regiter] as follows:

```
regex_iterator& operator++();  
regex_iterator operator++(int) = default;
```

[...]

```
regex_iterator operator++(int);
```

⁹ *Effects:* As if by:

```
regex_iterator tmp = *this;  
++(*this);  
return tmp;
```

Modify §28.6.11.2 [re.tokiter] as follows:

```
regex_token_iterator& operator++();  
regex_token_iterator operator++(int) = default;
```

[...]

```
regex_token_iterator& operator++(int);
```

⁹ *Effects:* Constructs a copy tmp of *this, then calls ++(*this).

¹⁰ *Returns:* tmp

Modify §29.10.6.X [simd.iterator] as follows:

```
constexpr simd_iterator& operator++();  
constexpr simd_iterator operator++(int) = default;  
constexpr simd_iterator& operator--();  
constexpr simd_iterator operator--(int) = default;
```

[...]

```
constexpr simd_iterator operator++(int);
```

⁵ *Effects:* Equivalent to:

```
simd_iterator tmp = *this;  
*this += 1;  
return tmp;
```

[...]

```
constexpr simd_iterator operator--(int);
```

⁷ *Effects:* Equivalent to:

```
simd_iterator tmp = *this;  
*this -= 1;  
return tmp;
```

3.2. Wording for Operators with Preconditions or Constraints

Modify §24.6.2.1 [istream.iterator] as follows:

```
istream_iterator& operator++();  
istream_iterator operator++(int) = default;
```

[...]

```
istream_iterator operator++(int);
```

⁸Preconditions: `in_stream != nullptr` is true.

⁹Effects: Equivalent to:

```
istream_iterator tmp = *this;  
++*this;  
return tmp;
```

Modify §29.10.6.1 [simd.class] as follows:

```
constexpr basic_mask& operator++() noexcept;  
constexpr basic_mask operator++(int) noexcept = default;  
constexpr basic_mask& operator--() noexcept;  
constexpr basic_mask operator--(int) noexcept = default;
```

[...]

```
constexpr basic_mask operator++(int) noexcept;
```

⁵Constraints: requires (value_type a) { a++; } is true.

⁶Effects: Increments every element by one.

⁷Returns: A copy of *this before incrementing.

[...]

```
constexpr basic_mask operator--(int) noexcept;
```

¹¹Constraints: requires (value_type a) { a--; } is true.

¹²Effects: Decrements every element by one.

¹³Returns: A copy of *this before decrementing.

3.3. Wording for Operators with Compile-Time Branching

Modify §25.5.4 [move.iterators] as follows:

```
constexpr move_iterator& operator++();  
constexpr autovoid operator++(int);  
constexpr move_iterator operator++(int) requires forward_iterator<Iterator> = default;
```

[...]

```
constexpr void operator++(int);
```

²Effects: If `Iterator` models `forward_iterator`, equivalent to:

```
move_iterator tmp = *this;  
++*this;  
return tmp;
```

Otherwise, equivalent to `++current`.

³Effects: Equivalent to `++current`.

Modify §25.7.16.3 [range.lazy.split.outer] as follows:

```
constexpr decltype(auto) operator++(int) {  
if constexpr (forward_range<Base>) {  
    auto tmp = *this;  
    ++*this;  
    return tmp;  
} else  
    ++*this;  
}  
  
constexpr void operator++(int) {  
    ++*this;  
}  
constexpr outer-iterator operator++(int) requires forward_range<Base> = default;
```

Modify §25.7.16.5 [range.lazy.split.inner] as follows:

```
constexpr decltype(auto) operator++(int) {  
if constexpr (forward_range<Base>) {  
    auto tmp = *this;  
    ++*this;  
    return tmp;  
} else  
    ++*this;  
}  
  
constexpr void operator++(int) {  
    ++*this;  
}  
constexpr inner-iterator operator++(int) requires forward_range<Base> = default;
```

3.4. Wording for Operators with both Preconditions and Compile-Time Branching:

Modify §24.5.5.1 [iterators.common] as follows:

```
constexpr common_iterator& operator++();
constexpr decltype(auto) operator++(int);
constexpr common_iterator operator++(int) requires forward_iterator<I> = default;

[...]
```

constexpr decltype(auto) operator++(int);

⁴Preconditions: `in_stream != nullptr` is true.

⁵Effects: ~~If `I` models `forward_iterator`, equivalent to:~~

```
common_iterator tmp = *this;
++*this;
return tmp;
```

~~Otherwise, if `I` requires(`I& i`) { { `*i++` } -> `can-reference`; } is true or~~

~~indirectly_readable<I> && constructible_from<iter_value_t<I>, iter_reference_t<I>> &&
move_constructible<iter_value_t<I>>~~

~~is false, equivalent to:~~

```
return get<I>(v_++);
```

~~Otherwise, equivalent to:~~

```
postfix-proxy p(*this);
++*this;
return p;
```

Where *postfix-proxy* is the exposition-only class:

```
class postfix-proxy {
    iter_value_t<I> keep_;
    constexpr postfix-proxy(iter_reference_t<I>&& x)
        : keep_(std::forward<iter_reference_t<I>>(x)) {}
public:
    constexpr const iter_value_t<I>& operator*() const noexcept {
        return keep_;
    }
};
```

3.5. Wording for Operators with Transitive Preconditions

Modify §24.5.7.1 [iterators.counted] as follows:

```
constexpr counted_iterator& operator++();  
constexpr decltype(auto) operator++(int);  
constexpr counted_iterator operator++(int)  
    requires forward_iterator<I> = default;  
  
[...]  
  
constexpr counted_iterator operator++(int) requires forward_iterator<I>  
5Effects: Equivalent to:  
  
    counted_iterator tmp = *this;  
    ++*this;  
    return tmp;
```

Modify §25.6.5.3 [range.repeat.iterator] as follows:

```
constexpr iterator& operator--();  
constexpr iterator operator--(int) = default;  
  
[...]  
  
constexpr iterator operator--(int);  
4Effects: Equivalent to:  
  
    auto tmp = *this;  
    --*this;  
    return tmp;
```

Modify §25.7.9.7 [range.chunk.fwd.iter] as follows:

```
constexpr iterator& operator++();  
constexpr iterator operator++(int) = default;  
  
[...]  
  
constexpr iterator operator++(int);  
4Effects: Equivalent to:  
  
    auto tmp = *this;  
    ++*this;  
    return tmp;
```

Modify §25.7.18.3 [range.concat.iterator] as follows:

```
constexpr iterator& operator++();
constexpr void operator++(int);
constexpr iterator operator++(int)
    requires all-forward<Const, Views...> = default;
constexpr iterator& operator--()
    requires concat-is-bidirectional<Const, Views...>;
constexpr iterator operator--(int)
    requires concat-is-bidirectional<Const, Views...> = default;

[...]
```

~~constexpr iterator operator++(int) requires all forward<Const, Views...>;~~
~~¹⁵Effects: Equivalent to:~~

```
auto tmp = *this;  
++*this;  
return tmp;
```

[...]

~~constexpr iterator operator--(int) requires concat is bidirectional<Const, Views...>;~~
~~¹⁶Effects: Equivalent to:~~

```
auto tmp = *this;  
--*this;  
return tmp;
```

Modify §25.7.27.3 [range.adjacent.iterator] as follows:

```
constexpr iterator& operator++();
constexpr iterator operator++(int) = default;

constexpr iterator& operator--() requires bidirectional_range<Base>;
constexpr iterator operator--(int) requires bidirectional_range<Base> = default;

[...]
```

~~constexpr iterator operator++(int);~~
~~¹⁰Effects: Equivalent to:~~

```
auto tmp = *this;  
++*this;  
return tmp;
```

[...]

~~constexpr iterator operator--(int) requires bidirectional_range<Base>;~~
~~¹¹Effects: Equivalent to:~~

```
auto tmp = *this;  
--*this;  
return tmp;
```

Modify §25.7.30.3 [range.slide.iterator] as follows:

```
constexpr iterator& operator++();
constexpr iterator operator++(int) = default;

constexpr iterator& operator--() requires bidirectional_range<Base>;
constexpr iterator operator--(int) requires bidirectional_range<Base> = default;

[...]

constexpr iterator operator++(int);
10Effects: Equivalent to:

auto tmp = *this;
++*this;
return tmp;

[...]

constexpr iterator operator--(int) requires bidirectional_range<Base>;
11Effects: Equivalent to:

auto tmp = *this;
--*this;
return tmp;
```

Modify §25.7.31.3 [range.chunk.by.iter] as follows:

```
constexpr iterator& operator++();
constexpr iterator operator++(int) = default;

[...]

constexpr iterator operator++(int);
10Effects: Equivalent to:

auto tmp = *this;
++*this;
return tmp;
```

References

Informative References

[P3480]

Matthias Kretz. *std::simd is a range*. URL: <https://wg21.link/P3480>

[P3668]

Matthew Taylor, Alex. *Defaulting Postfix Increment and Decrement Operations*. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3668r1.html>

[P3691]

Matthias Kretz, Abhilash Majumder, Bryce Adelstein Lelbach, Daniel Towner, Ilya Burylov, Mark Hoemmen, Ruslan Arutyunyan. *Reconsider naming of the namespace for “std::simd”*. URL: <https://wg21.link/P3691>